

# Cost-Aware Verification in Velvet via Syntax-Directed Instrumentation

Ronald Huidrom

May 14, 2026

## **Abstract**

Velvet is a Dafny-style verifier for imperative programs in Lean, built on Loom. This thesis extends Velvet with cost-aware verification, enabling programmers to specify and verify resource bounds alongside functional correctness. The extension is implemented via syntax-directed instrumentation using Lean’s elaboration mechanism, allowing cost reasoning to reuse the existing verification infrastructure. We demonstrate the approach on some examples.

# Contents

|          |                                            |          |
|----------|--------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                        | <b>2</b> |
| 1.1      | Introduction . . . . .                     | 5        |
| <b>2</b> | <b>A foundational multi-modal verifier</b> | <b>7</b> |
| 2.1      | Framework . . . . .                        | 7        |
| 2.2      | Shallow Embedding . . . . .                | 8        |
| 2.3      | Hybrid Workflow . . . . .                  | 8        |
| 2.4      | Velvet . . . . .                           | 8        |
| 2.5      | Imperative Programming Syntax . . . . .    | 9        |
| 2.6      | Multi-modal Verification . . . . .         | 10       |
| 2.7      | More on Loom and Velvet . . . . .          | 10       |

# Chapter 1

## Introduction

Consider the following function definition:

```
def divide (x : ℕ) (y : ℕ) (h : y ≠ 0) := x / y
```

This is the usual definition of a division function for natural numbers, except that instead of taking two arguments it takes three. By  $x : \mathbb{N}$ , we mean that  $x$  is a term having the type  $\mathbb{N}$  (the colon annotates the type). The third argument  $h$  has the type  $y \neq 0$ . This is a proposition and, furthermore, it depends on the value of the term  $y$ .

This is an instance of *dependent type theory*, in which types may depend on terms. By the Curry–Howard isomorphism and the *propositions-as-types, proofs-as-terms* viewpoint, a term of type  $y \neq 0$  is precisely a proof of that proposition. Therefore, the function `divide` can only be evaluated when we supply a proof that  $y$  is nonzero as its third argument.

A typical evaluation of the function would be:

```
#eval divide 10 2 (by decide)
```

Without going into details (which are delegated to later chapters), `by decide` constructs a term of the required type, in this case a proof of  $2 \neq 0$ .

Now consider the following code:

```
inductive NonEmptyList (α : Type) where
| mk : (xs : List α) → xs ≠ [] → NonEmptyList α
```

The keyword `inductive` creates an inductive data type named `NonEmptyList α`. Here,  $\alpha$  is an arbitrary type (for example, `String`), in which case `NonEmptyList String` denotes the type of non-empty lists of strings.

This inductive type has a single constructor `mk`. Outside the definition, it is referred to as `NonEmptyList.mk`, where the dot denotes a namespace (or simply `.mk` when the namespace can be inferred).

This means that there is exactly one way to construct a term of type `NonEmptyList α`, namely by using the constructor

```
NonEmptyList.mk : (xs : List α) → xs ≠ [] → NonEmptyList α
```

The type of this constructor can be understood as follows: it takes two arguments,

- a term `xs` of type `List α`, and
- a term of type `xs ≠ []`,

and returns a term of type `NonEmptyList α`.

The second argument is particularly interesting: it requires a proof that the first argument `xs` is non-empty. This is again an instance of dependent type theory, since the type `xs ≠ []` depends on the term `xs`.

Now consider the following definition:

```
def NonEmptyList.head {α} : NonEmptyList α → α
| .mk (x :: _) _ => x
| .mk [] h => (h rfl).elim
```

This definition implements the well-known `head` function. Its type is

```
NonEmptyList α → α
```

meaning that it takes a non-empty list of elements of type `α` and returns an element of type `α`.

In the function body, we use pattern matching to destruct the possible cases of a term of type `NonEmptyList α`. Since such a term can only be constructed using its constructor (this follows from the inductive definition), the cases listed above are exhaustive.

The first case is when the list has the form `x :: _`, in which case `x` is clearly the first element of the list. (The underscore `_` acts as a wildcard matching the remainder of the list. For example, the list `[1,2,3]` is written as `1 :: [2, 3]`.)

The second case considers the empty list `[]`. However, in this case `h` is a proof that `[]` is non-empty. Examining the type of `NonEmptyList.mk` reveals that `h` has type `[] ≠ []`. This proposition is definitionally equal to `[] = [] → False`. Since `rfl` is a proof of `[] = []`, the expression `h rfl` yields a contradiction, that is, a term of type `False`.

The expression `(h rfl).elim` invokes `False.elim`, which expresses the logical principle that from a contradiction, anything follows. This allows us to construct a term of type  $\alpha$ , thereby convincing Lean that this case is impossible.

All the code discussed above is written in Lean. We have seen that the type of a function can already guarantee its correctness: impossible cases are ruled out by logical reasoning. By allowing types to depend on runtime values, we eliminate entire classes of runtime errors.

Lean is both a general-purpose functional programming language and an interactive theorem prover, based on the calculus of inductive constructions and dependent type theory.

## 1.1 Introduction

Consider the following function definition:

```
def divide (x : ℕ) (y : ℕ) (h : y ≠ 0) := x / y
```

The declaration above is the usual integer division operation on natural numbers, except that the function takes an additional *proof* argument. By  $x : \mathbb{N}$  we mean that  $x$  is a term of type  $\mathbb{N}$ ; the third parameter  $h$  has type  $y \neq 0$ , a proposition that depends on the value of  $y$ . In other words, the function is not just *typed* but *indexed* by a proposition: it is a dependent function.

Under the Curry–Howard correspondence (propositions-as-types, proofs-as-terms), a term of type  $y \neq 0$  is exactly a witness (a proof) that  $y$  is nonzero. Thus `divide` can only be applied when the caller supplies such a witness. For example:

```
#eval divide 10 2 (by decide)
```

Here `by decide` constructs the required proof term (a certificate that  $2 \neq 0$ ), and the evaluation proceeds without any runtime check.

Now consider this inductive type:

```
inductive NonEmptyList (α : Type) where
| mk : (xs : List α) → xs ≠ [] → NonEmptyList α
```

The `inductive` declaration defines `NonEmptyList α` as the type of non-empty lists over  $\alpha$ . Its single constructor, `NonEmptyList.mk`, has the dependent type

```
NonEmptyList.mk : (xs : List α) → xs ≠ [] → NonEmptyList α
```

which reads: given a list  $xs : \text{List } \alpha$  and a proof of  $xs \neq []$ , construct an element of `NonEmptyList α`. The second argument is a *proof obligation* that enforces the invariant “the list is not empty” at the type level.

We can now implement a safe head projection:

```
def NonEmptyList.head {α} : NonEmptyList α → α
| .mk (x :: _) _ => x
| .mk [] h => (h rfl).elim
```

The pattern match is exhaustive because every `NonEmptyList α` is built with `.mk`. In the first clause the payload has the form  $x :: \_$  and we return the head  $x$ . The second clause is logically impossible: it matches the empty list `[]` while  $h$  is a proof of  $[] \neq []$ . Applying  $h$  to `rfl` (the canonical proof of judgmental equality  $[] = []$ ) yields a term of type `False`; invoking `(h rfl).elim` eliminates the contradiction (by

`False.elim`) and thus produces a term of the expected return type. In short, the impossible branch is discharged by logical contradiction rather than by a runtime error.

These examples illustrate a central design point of dependent-type proof assistants: invariants that would otherwise require runtime checks are encoded in the type system and discharged by supplying proof terms. By making propositions first-class and allowing types to depend on terms, Lean (based on the Calculus of Inductive Constructions) moves many correctness obligations from runtime into the type checker. The payoff is strong: programs and proofs can be composed so that certain classes of runtime failures are provably impossible.



# Chapter 2

## A foundational multi-modal verifier

### 2.1 Framework

Loom is a foundational verifier in that all its semantics and proofs are expressed in a theorem prover and proof assistant (Lean). Therefore, in this sense, final guarantees rely only on the Lean kernel. This shifts the burden of trust from an external software or language like Dafny to a theorem prover, thereby avoiding trusting large external verification pipelines.

To be precise, Loom is actually a framework for building multi-modal verifiers. That is to say, it is not a single verifier but rather a meta-framework for constructing many. But by abuse of language (and for convenience), we shall simply refer to Loom as a verifier to mean one such verifier built upon Loom. Later, we shall see Velvet which is an actual verifier built on top of Loom.

Loom provides some mathematical constructions and proofs of these constructions expressed in Lean. Programs in Loom are modeled using monads. Monads are a way to structure computations as a sequence of steps, where each step can also be seen as a computation that potentially gives a value while also producing potentially a side effect such as IO operation, failure, file reading or writing. Monads give a compositional semantics. Programs become mathematical objects inside Lean.

Loom uses an algebraic structure called monad transformer. Complex program behaviors can be built using monad transformers. A monad transformer is like a type constructor except it receives a monad and give back another monad. In our case, we think of each monad transformer encoding some effect such as failure or nondeterminism. By stacking monad transformers, one can then build programs

with multiple side effects. Verifiers can be constructed by stacking transformers.

## 2.2 Shallow Embedding

Lean is a very malleable language. Indeed, it is quite feasible to extend Lean or build a DSL inside Lean. Generally speaking, there are two approaches to building a language on the top of Lean. They are deep and shallow embedding.

In deep embedding, we use data structures (or inductive types in our case) to build abstract syntax tree (AST) and write interpreters to interpret them. If we go along this route to build our verifier language, we also have to verify the correctness of the semantics of our interpreter.

Loom instead uses shallow embedding. Here programs are written directly in Lean’s language. No separate syntax or external language is needed. Benefits are that it reuses Lean’s type system and it now has direct access to proofs and tactics. Programs are just Lean terms and this allows smooth integration with theorem proving.

## 2.3 Hybrid Workflow

Another strong usage of Loom would be its hybrid workflow. It aims to seamlessly blend automation via SMT solving with interactive theorem proving via Lean’s by-tactic blocks. This points to a unified framework for program execution, verification and proof development in the same environment.

## 2.4 Velvet

We have mentioned before that Loom is not a single verifier but rather a framework for building verifiers. We have been loosely referring to Loom as if it were an actual verifier language but now we meet Velvet, an actual DSL and verifier built on top of Lean.

The syntax of Velvet is inspired by Dafny. As with other verification languages, Velvet is used to write programs with specifications. Programs are written in imperative style (Velvet gives this imperative syntax). Velvet is also the name of the monad in which all programs are written. Indeed, Velvet is an abbreviation defined by

```
abbrev VelvetM  $\alpha$  := NonDetT DivM  $\alpha$ 
```

That is, `VelvetM` is a monad built from the divergence monad `DivM` (this encodes programs which can terminate or diverge, that is, to not terminate) and the non-determinism monad transformer `NonDetT` (this encodes programs which can have non-deterministic behavior).

Indeed, all programs in Velvet DSL have the type `VelvetM`.

Velvet combines automation (via `loom_solve` which is an automation tactic provided by Loom) and interactive proofs (which are just Lean tactics). More automation is possible through backends SMT solvers such as Z3 or CVC5. This gives a hybrid workflow in which verification usually proceeds by attacking with automation and followed by, if needed, more interactive theorem proving (tactics).

## 2.5 Imperative Programming Syntax

A majority of programmers are used to imperative-style programming like C or C++ or Java, and a lot of programming in current industry is mostly imperative. Velvet gives an imperative-style syntax built on top of the functional language Lean, made possible by leveraging the metaprogramming and elaboration techniques of the language.

The following is a typical program written in Velvet.

```
method DoubleAll (mut arr: Array Int) return (u: Unit)
  ensures arr.size = arrOld.size
  ensures  $\forall i, 0 \leq i \wedge i < \text{arr.size} \rightarrow \text{arr}[i]! = 2 * \text{arrOld}[i]!$ 
do
  let mut i := 0
  while i < arr.size
    invariant arr.size = arrOld.size
    invariant  $0 \leq i \wedge i \leq \text{arr.size}$ 
    invariant  $\forall k, 0 \leq k \wedge k < i \rightarrow \text{arr}[k]! = 2 * \text{arrOld}[k]!$ 
    invariant  $\forall k, i \leq k \wedge k < \text{arr.size} \rightarrow \text{arr}[k]! = \text{arrOld}[k]!$ 
    done_with i = arr.size
  do
    arr := arr.set! i (2 * arr[i]!)
    i := i + 1
  return
```

The function starts with the keyword `method`. The preconditions and postconditions are provided using `require` and `ensures` clauses. Multiple specifications are provided with multiple `require` and `ensures` clauses. The `while` loop requires invariants which are provided using `invariant` clause.

All functions written this way (using `method`) are elaborated to standard Lean programs or terms in the monad `VelvetM`. Indeed, if we type `#print DoubleAll`, we see the output:

```
def DoubleAll : Array Z → VelvetM (Unit × Array Z)
```

The above type can look a bit confusing (the extra `Array Z` in the return) but this is due to how mutable parameters are handled in Velvet which we will see later.

## 2.6 Multi-modal Verification

To execute or run the function, we simply evaluate:

```
#eval (DoubleAll #[1, 2, 3]).run
```

One can also perform property-based testing (PBT). This allows us to test methods with random inputs before actual verification.

To prove the functional correctness and that the specifications hold for all inputs, we do

```
prove_correct DoubleAll
  by loom_solve
```

As mentioned above, `loom_solve` is an automation tactic and usually forms the first line of attack. For most simple proof obligations, Velvet claims, this tactic is enough to finish the proof by itself. In the cases it does not, one can try more tactics to finish the remaining goals.

By multi-modal, it refers to the fact that Velvet can handle program execution, testing and deductive reasoning (formal verification) all in one environment.

## 2.7 More on Loom and Velvet

Velvet can do a lot more than what we have described in the above. For instance, it allows programmers to change the semantics (that is from “partial” to “total” correctness for termination, or from “demonic” to “angelic” for non-deterministic programs). These options are set in the beginning of the program file. Usually, the options are set to “partial correctness” and “demonic” choice.

Switching from “partial” to “total” correctness restricts programmers to write only terminating programs in the sense that when they write loops, for instance, they also have to provide termination measures.

For more details, one can refer to this excellent documentation. [\[HERE\]](#)

# Chapter 3

## Cost-Aware Verification

Functional correctness